Internship Training report

On

“DEVOPS & AWS CLOUD”

Submitted in partial fulfillment of the requirements for the award of Degree of

Masters of Computer Application

IN

DEPARTMENT OF COMPUTER APPLICATIONS

SESSION - 2024-26

Submitted By: Naveen Suthar
Roll Number : 24CEBMC614
Semester : IV
Department : MCA

Submitted To: Dr. Sanjay Tejasvee
HOD of MCA
Engineering College of Bikaner (Raj.)
334004

Engineering College of Bikaner Pugal Road
RIICO Karni Industrial Area, Bikaner
Rajasthan-334004 Telephone :
91-0151-2253404 Fax : +91-0151-2252919

Bikaner Technical University Pugal Road
RIICO Karni Industrial Area, Bikaner
Rajasthan-334004 Telephone :
0151-2250950 Fax : 0151-225094050

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Mr. Narendra Singh Bhui, Director, Centre for Cloud & DevOps Training Society (CDTS), Bikaner, for providing me the opportunity to undergo this 45-day Industrial Training cum Internship Programme on "DevOps & AWS Cloud". His guidance, mentorship, and support throughout the training period have been invaluable in shaping my understanding of modern DevOps practices and cloud technologies.

I am deeply thankful to the entire team at CDTS, Bikaner for creating a practical, hands-on learning environment covering Linux, Bash Scripting, Python, Git, Docker, CI/CD Pipelines, Ansible, Kubernetes, and Amazon Web Services.

I extend my heartfelt thanks to the faculty members of the Department of Computer Application, Engineering College Bikaner, affiliated with Bikaner Technical University, for their constant encouragement and academic support throughout my MCA programme.

Special thanks to my family and friends for their unwavering support and motivation during the course of this training.

Nikhil Pareek
MCA Semester 4th
Engineering College Bikaner
Roll No.: 24CEBMC614

DECLARATION

I, Nikhil Pareek, student of Master of Computer Applications (MCA) Final Year at Engineering College Bikaner, affiliated with Bikaner Technical University (BTU), Rajasthan, hereby declare that this Industrial Training cum Internship Report entitled "DevOps & AWS Cloud — A Comprehensive Training Report" has been prepared by me based on the 45-day internship programme undergone at Centre for Cloud & DevOps Training Society (CDTS), Bikaner, from 19.01.2026 to 05.03.2026.

This report is an original work and has not been submitted elsewhere for any academic purpose. The content presented in this report is based on the knowledge acquired through the training, practical implementation, and self-study of the DevOps tools and technologies covered during the internship.

All information, data, and references used in this report have been duly acknowledged. Any resemblance to previously published material is purely coincidental.

Signature: _____

Nikhil Pareek
MCA Semester 4th
Engineering College Bikaner
Roll No.: 24CEBMC615

INDEX

S.No	Title	Page No.
1	Acknowledgement	1
2	Declaration	2
3	Certificate of Completion	3
4	Company Certificate — To Whom It May Concern	4
5	Index	5
6	Training Overview — 45-Day Programme at CDTS, Bikaner	6-7
7	Module 1: Linux Fundamentals	8-13
8	Module 2: Bash Scripting	14-7
9	Module 3: Python for DevOps	18-20
10	Module 4: Git & Version Control	21-23
11	Module 5: Docker & Containerization	24-26
12	Module 6: CI/CD Pipelines	27-29
13	Module 7: Ansible	30-32
14	Module 8: Kubernetes	33-36
15	Module 9: Amazon Web Services (AWS)	37-40
16	Module 10: DevOps Culture & SRE Principles	41-42
17	Appendix A: Command Cheatsheet	43-44
18	Appendix B: Glossary	45

Training Overview — 45-Day Programme at CDTS, Bikaner

About CDTS

Centre for Cloud & DevOps Training Society (CDTS), Bikaner is a registered IT training organisation (Society Reg. No. COOP/2020/BIKANER/200308) specialising in open-source technologies, cloud computing, and DevOps training. Located at Amar Singh Pura, Behind Rajasthan Patrika Press, Bikaner — 334001, CDTS offers industry-aligned training programmes for engineering and MCA students, with a focus on hands-on practical exposure to tools used in modern software development and infrastructure management.

Programme Details

Detail	Information
Programme	Industrial Training cum Internship — DevOps & AWS Cloud
Organisation	Centre for Cloud & DevOps Training Society (CDTS), Bikaner
Director	Narendra Singh Bhui
Duration	45 Days
Certificate No.	CDTS/CERT/Gen-File/255
F. No.	/CDTS/2026/MCA-Internship/155
Student	Nikhil Pareek
Roll No.	24CEBMC615
Programme	Master of Computer Applications (MCA) Final Year
Institute	Engineering College Bikaner, affiliated to Bikaner Technical University

45-Day Week-wise Training Schedule

Week	Days	Topics Covered
Week 1	Day 1–7	Linux fundamentals — architecture, filesystem hierarchy (FHS), essential commands, file permissions, user & group management
Week 2	Day 8–14	Linux advanced — process management, networking commands, package management (APT/YUM), disk & storage management, text processing tools (awk, sed, grep)
Week 3	Day 15–21	Bash scripting — variables, operators, conditionals, loops, functions, arrays, string manipulation, file operations, cron jobs, practical automation scripts

Week 4	Day 22–28	Python for DevOps — file handling, OS & subprocess modules, API interaction (requests), environment variables, boto3 basics; Git & version control — branching strategies, remote operations, GitHub Actions introduction
Week 5	Day 29–35	Docker & containerization — architecture, images, containers, Dockerfile best practices, Docker Compose, volumes, networking, multi-stage builds, image optimisation
Week 6	Day 36–42	CI/CD pipelines — GitHub Actions workflows, pipeline stages, secrets management; Ansible — inventory, playbooks, roles, Jinja2 templates; Kubernetes — architecture, Pods, Deployments, Services, ConfigMaps, kubectl, Helm
Week 6.5	Day 43–45	Amazon Web Services — IAM, EC2, S3, VPC, RDS, Lambda, CloudWatch, AWS CLI; DevOps Culture & SRE principles — Three Ways, DORA metrics, SLI/SLO/SLA, error budgets

Technologies and Tools Covered

Category	**Tools / Technologies**
Operating System	Linux (Ubuntu, CentOS)
Shell & Scripting	Bash, Cron, Shell utilities (awk, sed, grep, find)
Programming	Python 3 (os, subprocess, requests, boto3)
Version Control	Git, GitHub, GitHub Actions
Containerization	Docker, Docker Compose
CI/CD	GitHub Actions, Pipeline automation
Configuration Mgmt.	Ansible, Jinja2 templates, Playbooks, Roles
Container Orchestration	Kubernetes (kubectl), Helm
Cloud Platform	Amazon Web Services — IAM, EC2, S3, VPC, RDS, Lambda, CloudWatch
DevOps Principles	SRE, DORA metrics, SLI/SLO/SLA, error budgets, blameless post-mortems

Module 1: Linux Fundamentals

1.1 Introduction to Linux

Linux is a free and open-source operating system kernel first created by Linus Torvalds in 1991. Built on the Unix philosophy, it has become the backbone of modern computing infrastructure. Linux powers the majority of web servers, cloud platforms, supercomputers, and embedded systems worldwide. It is distributed under the GNU General Public License (GPL), allowing users to view, modify, and distribute the source code freely.

1.1.1 Why Linux for DevOps

- Stability and reliability — Linux servers can run for years without requiring a reboot.
- Security — fine-grained permission model, SELinux, and AppArmor provide strong security controls.
- Performance — minimal overhead makes Linux ideal for running servers and containers.
- Scripting and automation — powerful shell and scripting capabilities through Bash and other shells.
- Container compatibility — Docker and Kubernetes are built natively on Linux kernel features such as cgroups and namespaces.
- Cost — no licensing fees; the operating system and most tools are free.

1.1.2 Popular Linux Distributions

Distribution	Description
Ubuntu	Debian-based; widely used for development and servers; LTS releases every 2 years
CentOS / RHEL	Enterprise-grade; used in production environments; RPM-based package management
Debian	Highly stable; forms the base for Ubuntu; preferred for servers
Fedora	Cutting-edge features; Red Hat sponsored; good for developers
Alpine Linux	Extremely lightweight; widely used as a base for Docker containers
Amazon Linux 2	AWS-optimised distribution; used on EC2 instances

1.2 Linux Architecture

Linux follows a layered architecture consisting of the hardware layer, kernel, shell, and user applications.

Hardware Layer:

Physical components — CPU, RAM, storage devices, and network interfaces. The kernel communicates with hardware via device drivers.

Kernel:

Core of the operating system. Manages CPU scheduling, memory management, I/O operations, process management, and inter-process communication. The Linux kernel is monolithic but supports loadable kernel modules.

Shell:

Interface between the user and the kernel. Interprets user commands and passes them to the kernel. Common shells include Bash, Zsh, Fish, and Dash.

User Space / Applications:
Programs and utilities that run outside the kernel — text editors, compilers, web servers, databases, and DevOps tools.

1.3 Filesystem Hierarchy Standard (FHS)

Linux organises the filesystem in a tree structure starting from the root directory /. Every file and directory is located under this root.

Directory	Purpose
/	Root directory — top-level of the entire filesystem hierarchy
/bin	Essential user command binaries (ls, cp, mv, cat)
/sbin	System administration binaries (fdisk, iptables, reboot)
/etc	Configuration files for the system and installed software
/home	Home directories for regular users
/root	Home directory of the root (superuser) account
/var	Variable data — logs (/var/log), mail, print spools
/tmp	Temporary files cleared on reboot
/usr	User programs, libraries, and documentation
/opt	Optional third-party software packages
/proc	Virtual filesystem exposing kernel and process information
/dev	Device files representing hardware components
/mnt	Temporary mount points for external filesystems
/lib	Shared libraries required by /bin and /sbin binaries

1.4 Essential Linux Commands

1.4.1 Navigation Commands

Command	Description
pwd	Print current working directory
ls	List directory contents
ls -la	List all files including hidden, with detailed info
cd /path	Change directory to specified path

cd ~	Navigate to home directory
cd ..	Move one level up in directory tree

1.4.2 File and Directory Operations

Command	Description
touch file.txt	Create an empty file
mkdir dirname	Create a new directory
mkdir -p a/b/c	Create nested directories
cp src dest	Copy file from source to destination
cp -r src/ dest/	Recursively copy a directory
mv src dest	Move or rename a file/directory
rm file.txt	Remove a file
rm -rf dirname/	Forcefully remove a directory and contents
find / -name 'file.txt'	Search for a file by name
locate filename	Quickly locate files using indexed database

1.4.3 File Viewing and Editing

Command	Description
cat file.txt	Display entire file contents
less file.txt	View file with scroll support
head -n 20 file.txt	Show first 20 lines of a file
tail -n 20 file.txt	Show last 20 lines of a file
tail -f logfile	Follow file output in real time (useful for logs)
grep 'pattern' file	Search for a pattern in a file
grep -r 'text' dir/	Recursively search in directory
wc -l file.txt	Count number of lines in a file
diff file1 file2	Compare two files line by line
vim filename	Open file in Vim editor
nano filename	Open file in Nano editor

1.5 File Permissions and Ownership

Linux uses a permissions model based on three categories: Owner, Group, and Others — with three types of permissions: Read (r), Write (w), and Execute (x). Each file has a 10-character permission string, for example: -rwxr-xr--

Symbol	Meaning
-	File type: - for file, d for directory, l for symbolic link
rwx	Owner permissions: read, write, execute
r-x	Group permissions: read, no write, execute
r--	Others permissions: read only

chmod — Change Permissions

```
chmod 755 script.sh       # rwxr-xr-x
chmod 644 config.txt      # rw-r--r--
chmod +x script.sh        # Add execute permission
chmod -w file.txt         # Remove write permission
```

chown — Change Ownership

```
chown nikhil file.txt                # Change owner
chown nikhil:devops file.txt         # Change owner and group
chown -R nikhil:devops /var/www/     # Recursive ownership change
```

1.6 User and Group Management

Command	Description
useradd username	Create a new user account
passwd username	Set or update a user's password
usermod -aG group user	Add user to a group
userdel username	Delete a user account
groupadd groupname	Create a new group
id username	Show user ID and group memberships
whoami	Print current logged-in username
su - username	Switch to another user
sudo command	Execute command with superuser privileges
visudo	Safely edit the sudoers configuration file

1.7 Process Management

Command	Description
ps aux	List all running processes with details
top	Real-time process viewer (CPU, memory usage)
htop	Enhanced interactive process viewer
kill PID	Send SIGTERM to process by its PID
kill -9 PID	Force kill a process (SIGKILL)
pkill processname	Kill process by name
nohup command &	Run a command that persists after logout
systemctl start svc	Start a systemd service
systemctl stop svc	Stop a systemd service
systemctl status svc	Check status of a systemd service
systemctl enable svc	Enable service to start at boot

1.8 Networking Commands

Command	Description
ip addr	Display network interfaces and IP addresses
ping hostname	Test network connectivity
curl URL	Transfer data from a URL (HTTP/HTTPS)
wget URL	Download files from the web
netstat -tulnp	Show active ports and listening services
ss -tulnp	Modern replacement for netstat
ssh user@host	Connect to remote host via SSH
scp file user@host:/path	Securely copy file to remote host
rsync -avz src dest	Sync files between local/remote locations
nslookup domain	Query DNS for a domain name
ufw allow 22	Allow SSH through UFW firewall

1.9 Package Management

APT (Debian/Ubuntu)

```
sudo apt update                    # Refresh package index
sudo apt upgrade                   # Upgrade all installed packages
sudo apt install package-name      # Install a package
sudo apt remove package-name       # Remove a package
```

```
sudo apt autoremove                # Remove unused dependencies
```

YUM / DNF (RHEL/CentOS/Fedora)

```
sudo yum update                    # Update packages
sudo yum install package-name      # Install a package
sudo dnf install package-name      # DNF is the modern replacement for YUM
```

1.10 Text Processing Tools

Command	Description
awk '{print $1}' file	Extract the first column from each line
sed 's/old/new/g' file	Replace all occurrences of 'old' with 'new'
sort file.txt	Sort lines of a file alphabetically
uniq file.txt	Remove duplicate consecutive lines
cut -d: -f1 /etc/passwd	Extract first field using : as delimiter
tee output.txt	Read stdin and write to both stdout and file
xargs	Build and execute commands from stdin

Module 2: Bash Scripting

2.1 Introduction to Bash

Bash (Bourne Again Shell) is the default command-line interpreter for most Linux distributions. It provides a scripting language that allows automation of repetitive tasks, system administration, file management, and application deployment. Bash scripting is a critical skill in DevOps for building automation pipelines, writing deployment scripts, and orchestrating system tasks.

```
#!/bin/bash
echo "Hello, DevOps!"
# Make script executable and run
chmod +x script.sh && ./script.sh
```

2.2 Variables

```
#!/bin/bash
NAME="Nikhil"
AGE=25
CURRENT_DIR=$(pwd)          # Command substitution
echo "Name: $NAME, Age: $AGE, Dir: $CURRENT_DIR"
```

Special Variables

Variable	Description
$0	Name of the script
$1..$9	Positional arguments passed to the script
$#	Number of arguments passed
$@	All arguments as a list
$?	Exit status of the last command (0 = success)
$$	PID of the current shell/script
$USER	Current logged-in username
$HOME	Home directory of current user
$PATH	Colon-separated list of directories for commands

2.3 Operators and Arithmetic

```
A=10; B=3
echo $((A + B))        # Addition: 13
echo $((A * B))        # Multiplication: 30
echo $((A % B))        # Modulo: 1
```

```
result=$(expr $A + $B)
```

2.4 Conditional Statements

```
#!/bin/bash
AGE=20
if [ $AGE -ge 18 ]; then
  echo "Adult"
elif [ $AGE -ge 13 ]; then
  echo "Teenager"
else
  echo "Child"
fi
```

Operator	Meaning
-eq	Equal to
-ne	Not equal to
-lt	Less than
-le	Less than or equal to
-gt	Greater than
-ge	Greater than or equal to
-z	String is empty
-f	File exists and is a regular file
-d	Directory exists
-e	File or directory exists

2.5 Loops

For Loop

```
for FRUIT in apple banana mango; do
  echo "Fruit: $FRUIT"
done

for i in {1..5}; do echo "Iteration: $i"; done

for ((i=0; i<5; i++)); do echo "Count: $i"; done
```

While Loop

```
COUNT=1
while [ $COUNT -le 5 ]; do
  echo "Count: $COUNT"
  ((COUNT++))
done
```

2.6 Functions and Arrays

```
greet() {
  local NAME=$1
  echo "Hello, $NAME!"
}
greet "Nikhil"

SERVERS=("web01" "web02" "db01")
echo ${SERVERS[0]}          # First element: web01
echo ${#SERVERS[@]}         # Array length: 3
for SERVER in ${SERVERS[@]}; do
  echo "Checking $SERVER"
done
```

2.7 Cron Jobs — Task Scheduling

Cron is a time-based job scheduler in Linux. Cron jobs are defined in a crontab file and allow tasks to run automatically at specified intervals.

```
# Cron syntax: minute hour day month weekday command
crontab -e

# Run backup every day at 2:00 AM
0 2 * * * /home/nikhil/backup.sh

# Run health check every 5 minutes
*/5 * * * * /opt/scripts/healthcheck.sh >> /var/log/health.log
```

2.8 Practical Script — Automated Backup

```
#!/bin/bash
BACKUP_DIR="/backup"
SOURCE_DIR="/var/www/html"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="$BACKUP_DIR/backup_$DATE.tar.gz"
mkdir -p $BACKUP_DIR
tar -czf $BACKUP_FILE $SOURCE_DIR
if [ $? -eq 0 ]; then
  echo "Backup successful: $BACKUP_FILE"
else
  echo "Backup failed!"; exit 1
fi
# Delete backups older than 7 days
find $BACKUP_DIR -name '*.tar.gz' -mtime +7 -delete
```

2.9 Practical Script — Server Health Check

```
#!/bin/bash
```

```
echo "=== Server Health Check ==="
echo "Hostname: $(hostname)"
echo "Uptime: $(uptime -p)"
CPU=$(top -bn1 | grep 'Cpu(s)' | awk '{print $2}' | cut -d'%' -f1)
echo "CPU Usage: $CPU%"
MEM=$(free -h | grep Mem | awk '{print $3 "/" $2}')
echo "Memory Usage: $MEM"
DISK=$(df -h / | tail -1 | awk '{print $5}')
echo "Disk Usage: $DISK"
```

Module 3: Python for DevOps

3.1 Introduction

Python is one of the most widely used programming languages in DevOps. Its simplicity, readability, and extensive standard library make it ideal for automation, infrastructure management, API interaction, and data processing. Tools like Ansible are written in Python, and major cloud SDKs (AWS boto3) provide Python-native interfaces.

3.2 Python Basics for DevOps

```
# Variables and Data Types
name = "DevOps"; version = 3.11; is_active = True
servers = ["web01", "web02", "db01"]
config = {"host": "localhost", "port": 5432}

# Control Flow
for server in servers:
  print(f"Checking {server}...")

status_code = 200
if status_code == 200: print("OK")
elif status_code == 404: print("Not Found")
else: print(f"Unexpected: {status_code}")
```

3.3 File Handling

```
# Read a file
with open('/etc/hostname', 'r') as f:
  hostname = f.read().strip()

# Write to a file
with open('deployment.log', 'a') as f:
  f.write('Deployment started\n')

# Read JSON config
import json
with open('config.json', 'r') as f:
```

```
  config = json.load(f)
```

3.4 OS and Subprocess Modules

The os and subprocess modules allow Python scripts to interact with the operating system, execute shell commands, and manage the filesystem — making them essential for DevOps automation.

```
import os, subprocess

# OS module
cwd = os.getcwd()
os.makedirs('/opt/app/logs', exist_ok=True)
env_var = os.environ.get('DATABASE_URL', 'localhost')

# subprocess — run shell commands
result = subprocess.run(['git', 'status'], capture_output=True, text=True)
print(result.stdout)

# Check if Docker is running
try:
  subprocess.run(['docker', 'info'], check=True, capture_output=True)
  print("Docker is running")
except subprocess.CalledProcessError:
  print("Docker is not running")
```

3.5 Working with APIs using requests

```
import requests

response = requests.get('https://api.example.com/health')
print(response.status_code)   # 200
data = response.json()

payload = {'service': 'auth', 'action': 'restart'}
headers = {'Authorization': 'Bearer TOKEN123'}
response = requests.post('https://api.example.com/services',
  json=payload, headers=headers)
```

3.6 Boto3 — AWS Automation with Python

Boto3 is the official AWS SDK for Python. It allows programmatic interaction with AWS services such as EC2, S3, RDS, Lambda, and more.

```
import boto3

# List S3 buckets
s3 = boto3.client('s3')
response = s3.list_buckets()
for bucket in response['Buckets']:
```

```
  print(bucket['Name'])

# Upload a file to S3
s3.upload_file('backup.tar.gz', 'my-bucket', 'backups/backup.tar.gz')

# List EC2 instances
ec2 = boto3.resource('ec2')
for instance in ec2.instances.all():
  print(instance.id, instance.state['Name'])
```

3.7 Log Analyser Script

```
#!/usr/bin/env python3
import re
from collections import Counter

def analyse_logs(filepath):
  error_pattern = re.compile(r'ERROR|CRITICAL|FATAL')
  ip_pattern = re.compile(r'\d+\.\d+\.\d+\.\d+')
  errors, ips = [], []
  with open(filepath) as f:
    for line in f:
      if error_pattern.search(line): errors.append(line.strip())
      ip = ip_pattern.search(line)
      if ip: ips.append(ip.group())
  print(f"Total errors: {len(errors)}")
  for ip, count in Counter(ips).most_common(5):
    print(f"  {ip}: {count} requests")

analyse_logs('/var/log/nginx/access.log')
```

Module 4: Git & Version Control

4.1 Introduction to Git

Git is a distributed version control system (DVCS) created by Linus Torvalds in 2005. It tracks changes in source code, enables collaboration among teams, and provides a complete history of every modification. In DevOps, Git is the foundation of CI/CD pipelines and collaborative development workflows.

4.2 Git Architecture

- Working Directory — where files are actively being edited.
- Staging Area (Index) — where changes are prepared before committing.
- Repository (.git) — the local database storing all commits and history.

4.3 Core Git Commands

Command	Description
git init	Initialise a new Git repository
git clone <url>	Clone a remote repository locally
git status	Show current state of working directory
git add .	Stage all changed files
git commit -m 'message'	Commit staged changes with a message
git log --oneline	View condensed commit history
git diff	Show unstaged changes
git stash	Temporarily save uncommitted changes
git stash pop	Restore the most recently stashed changes

4.4 Branching and Merging

Command	Description
git branch	List all local branches
git branch feature-x	Create a new branch
git checkout -b feature-x	Create and switch to a new branch
git merge feature-x	Merge feature-x into current branch

git rebase main	Rebase current branch onto main
git branch -d feature-x	Delete a merged branch

4.5 Branching Strategies

GitFlow:

Uses two main branches (main and develop) and supporting branches (feature, release, hotfix). Suited for projects with scheduled releases. main holds production-ready code only; develop is the integration branch.

Trunk-Based Development:

Developers commit directly to a single main branch in small, frequent increments. Feature flags hide incomplete features. This model supports continuous integration and deployment pipelines.

4.6 Remote Repository Operations

Command	Description
git remote add origin <url>	Link local repo to remote
git push origin main	Push changes to remote main branch
git pull origin main	Fetch and merge remote changes
git fetch	Download remote changes without merging
git push -u origin feature	Push branch and set upstream tracking

4.7 .gitignore

```
# Python
__pycache__/
*.pyc
.env
venv/

# Node.js
node_modules/
dist/

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
```

4.8 GitHub Actions — Introduction

GitHub Actions is a CI/CD platform integrated directly into GitHub. It allows automation of build, test, and deployment workflows triggered by events such as push, pull request, or schedule.

```
# .github/workflows/ci.yml
name: CI Pipeline
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run tests
        run: |
          pip install -r requirements.txt
          pytest
```

Module 5: Docker & Containerization

5.1 Introduction to Containers

Containers are lightweight, portable, and self-sufficient units that package an application along with its dependencies, libraries, and runtime. Unlike virtual machines, containers share the host operating system kernel, making them significantly more efficient in resource consumption and startup time.

Aspect	Containers	Virtual Machines
OS	Share host OS kernel	Full OS per VM
Size	Megabytes	Gigabytes
Startup Time	Seconds	Minutes
Isolation	Process-level (cgroups)	Hardware-level
Portability	Very high	Moderate

5.2 Docker Architecture

- Docker Client — the CLI tool (docker) that sends commands to the daemon.
- Docker Daemon (dockerd) — manages containers, images, networks, and volumes.
- Docker Registry — a repository for Docker images. Docker Hub is the default public registry.
- Docker Images — read-only templates used to create containers.
- Docker Containers — running instances of Docker images.

5.3 Essential Docker Commands

Command	Description
docker pull nginx	Download an image from Docker Hub
docker run -d -p 8080:80 nginx	Run container detached, map ports
docker run --name webserver nginx	Assign a name to the container
docker ps	List running containers
docker ps -a	List all containers including stopped
docker stop container_id	Gracefully stop a running container
docker rm container_id	Remove a stopped container
docker rmi image_id	Remove a Docker image
docker exec -it container bash	Open interactive bash shell in container

docker logs -f container_id	Follow container stdout/stderr logs
docker stats	Live container resource usage
docker system prune -af	Remove all unused Docker resources

5.4 Dockerfile

A Dockerfile is a text file containing instructions to build a Docker image. Each instruction creates a new layer in the image.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN addgroup --system appgroup && adduser --system appuser --ingroup appgroup
USER appuser
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Instruction	**Purpose**
FROM	Specifies the base image
WORKDIR	Sets the working directory inside the container
COPY	Copies files from host into the image
RUN	Executes a command during image build
ENV	Sets an environment variable
EXPOSE	Documents the port the container listens on
CMD	Default command to run when container starts
ENTRYPOINT	Main command that cannot be overridden easily
ARG	Build-time variables passed with --build-arg

5.5 Docker Volumes and Networking

Volumes

```
docker volume create dbdata
docker run -v dbdata:/var/lib/postgresql/data postgres
docker run -v /home/nikhil/app:/app myapp    # Bind mount
```

Networking

```
docker network create mynetwork
docker run --network mynetwork --name backend myapp
docker run --network mynetwork --name db postgres
```

5.6 Docker Compose

```
version: '3.9'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/appdb
    depends_on:
      - db
  db:
    image: postgres:15
    volumes:
      - dbdata:/var/lib/postgresql/data
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=appdb
volumes:
  dbdata:
```

Command	Description
docker compose up -d	Start all services in detached mode
docker compose down	Stop and remove containers
docker compose logs -f web	Follow logs of the web service
docker compose ps	List status of compose services
docker compose build	Build/rebuild service images

5.7 Image Optimisation — Multi-Stage Builds

```
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
```

Module 6: CI/CD Pipelines

6.1 Introduction to CI/CD

CI/CD stands for Continuous Integration, Continuous Delivery, and Continuous Deployment. It is a set of practices and automation techniques that enable development teams to deliver code changes more frequently and reliably. CI/CD is the backbone of modern DevOps culture, bridging the gap between development and operations.

- Continuous Integration (CI) — developers frequently merge code into a shared repository. Automated builds and tests run on every commit to detect issues early.
- Continuous Delivery (CD) — ensures the codebase is always in a deployable state. Production deployment requires a manual approval step.
- Continuous Deployment — every change that passes automated tests is automatically deployed to production without manual intervention.

6.2 CI/CD Pipeline Stages

Stage	Description
1. Source	Code is pushed to a version control system (Git)
2. Build	Application is compiled, assets bundled, Docker image built
3. Test	Unit tests, integration tests, linting, security scans
4. Staging Deploy	Artifact deployed to staging environment for validation
5. Acceptance Tests	End-to-end and performance tests against staging
6. Production Deploy	Artifact deployed to production environment
7. Monitor	Application monitored for errors and performance

6.3 GitHub Actions — Key Concepts

Concept	Description
Workflow	An automated process defined in a YAML file under .github/workflows/
Event	A trigger that starts a workflow (push, pull_request, schedule)
Job	A set of steps that run on the same runner
Step	An individual task — a shell command or a reusable Action
Action	A reusable unit of automation (e.g. actions/checkout)
Runner	The server where jobs execute (GitHub-hosted or self-hosted)

Secret	Encrypted environment variable stored in repository settings
Artifact	Files produced by a workflow (build outputs, test reports)

6.4 Complete Python CI/CD Pipeline

```
name: Python CI/CD
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest flake8
      - name: Lint
        run: flake8 . --max-line-length=120
      - name: Run tests
        run: pytest tests/ -v

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v3
      - name: Login to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}
      - name: Build and push
        run: |
          docker build -t myapp:${{ github.sha }} .
          docker push myapp:${{ github.sha }}

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    environment: production
    steps:
```

```
- name: Deploy via SSH
  uses: appleboy/ssh-action@v0.1.7
  with:
    host: ${{ secrets.SERVER_HOST }}
    username: ${{ secrets.SERVER_USER }}
    key: ${{ secrets.SSH_PRIVATE_KEY }}
    script: |
      docker pull myapp:${{ github.sha }}
      docker stop myapp || true
      docker run -d --name myapp -p 8000:8000 myapp:${{ github.sha }}
```

6.5 Pipeline Best Practices

- Keep pipelines fast — run unit tests first; slow integration tests last.
- Fail fast — stop the pipeline at the first failure to save resources.
- Cache dependencies — avoid re-downloading packages on every run.
- Immutable artifacts — build once, deploy the same artifact to all environments.
- Gate deployments — require manual approval for production deployments.
- Never hardcode secrets — always use encrypted secrets or environment variables.

Module 7: Ansible

7.1 Introduction to Ansible

Ansible is an open-source IT automation tool developed by Red Hat. It enables configuration management, application deployment, and task automation across multiple servers simultaneously. Ansible is agentless — it connects to managed nodes via SSH and executes tasks using Python. Playbooks are written in YAML, making them human-readable and easy to version-control.

7.2 Ansible Architecture

Component	Description
Control Node	The machine where Ansible is installed and from which commands are run
Managed Nodes	The remote servers managed by Ansible (no agent installation required)
Inventory	A file listing the managed nodes, grouped by role or environment
Modules	Reusable units of work (apt, copy, service, docker_container)
Playbooks	YAML files containing ordered lists of tasks to execute on managed nodes
Roles	A structured way to organise related tasks, variables, and files for reuse
Facts	System information automatically gathered from managed nodes

7.3 Inventory File

```
[webservers]
web01 ansible_host=192.168.1.10
web02 ansible_host=192.168.1.11

[databases]
db01 ansible_host=192.168.1.20

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

7.4 Ad-Hoc Commands

```
ansible all -m ping                                        # Ping all hosts
ansible webservers -m shell -a 'df -h'                     # Run shell command
ansible webservers -m apt -a 'name=nginx state=present' -b  # Install package
ansible webservers -m service -a 'name=nginx state=restarted' -b
```

7.5 Playbooks

```
---
- name: Install and configure Nginx
  hosts: webservers
  become: yes
  vars:
    nginx_port: 80
  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes
    - name: Install Nginx
      apt:
        name: nginx
        state: present
    - name: Start and enable Nginx
      service:
        name: nginx
        state: started
        enabled: yes
      notify: Restart Nginx
  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
```

7.6 Variables and Jinja2 Templates

```
# vars/main.yml
db_host: localhost
db_port: 5432

# templates/app.conf.j2
[database]
HOST = {{ db_host }}
PORT = {{ db_port }}

# Task
- name: Deploy app config
  template:
    src: templates/app.conf.j2
    dest: /etc/app/app.conf
```

7.7 Ansible Roles — Directory Structure

```
roles/
└── nginx/
    ├── tasks/main.yml
    ├── handlers/main.yml
    ├── templates/nginx.conf.j2
```

```
├── vars/main.yml
└── defaults/main.yml
```

Module 8: Kubernetes

8.1 Introduction to Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform originally developed by Google and now maintained by the Cloud Native Computing Foundation (CNCF). It automates the deployment, scaling, and management of containerised applications across clusters of machines, providing self-healing, automatic scaling, rolling updates, service discovery, and load balancing.

8.2 Kubernetes Architecture

Control Plane (Master Node)

Component	Description
kube-apiserver	Frontend for the Kubernetes control plane; all communication goes through the API server
etcd	Distributed key-value store; stores all cluster state and configuration
kube-scheduler	Assigns newly created pods to nodes based on resource availability
kube-controller-manager	Runs controllers that regulate the state of the cluster

Worker Nodes

Component	Description
kubelet	Agent running on each node; ensures containers in pods are running and healthy
kube-proxy	Maintains network rules on nodes; handles service traffic routing
Container Runtime	Software that runs containers (Docker, containerd, CRI-O)

8.3 Core Kubernetes Objects

Pods

A Pod is the smallest deployable unit in Kubernetes. It wraps one or more containers that share networking and storage.

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
```

```
  containers:
    - name: myapp
      image: myapp:1.0
      ports:
        - containerPort: 8000
      resources:
        requests:
          memory: '64Mi'
          cpu: '250m'
        limits:
          memory: '128Mi'
          cpu: '500m'
```

Deployments

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myapp:1.0
          ports:
            - containerPort: 8000
```

Services

Service Type	Description
ClusterIP	Exposes the service on an internal cluster IP (default)
NodePort	Exposes the service on each node's IP at a static port
LoadBalancer	Creates an external load balancer (cloud provider)
ExternalName	Maps the service to an external DNS name

8.4 ConfigMaps and Secrets

```
# ConfigMap
apiVersion: v1
```

```
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info

# Secret (values are base64-encoded)
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQxMjM=
```

8.5 kubectl — Essential Commands

Command	Description
kubectl get pods	List all pods in default namespace
kubectl get deployments	List all deployments
kubectl get services	List all services
kubectl get nodes	List cluster nodes
kubectl describe pod <name>	Detailed info about a pod
kubectl logs -f <pod>	Follow pod logs in real time
kubectl exec -it <pod> -- bash	Interactive shell in a running pod
kubectl apply -f manifest.yaml	Apply a YAML manifest to the cluster
kubectl delete -f manifest.yaml	Delete resources from a manifest
kubectl scale deployment app --replicas=5	Scale a deployment
kubectl rollout undo deploy/app	Roll back to previous deployment
kubectl top pods	Resource usage by pod

8.6 Helm — Kubernetes Package Manager

Helm is the package manager for Kubernetes. It uses charts — pre-configured templates of Kubernetes resources — to simplify deployment of complex applications.

Command	Description
helm install myapp ./chart	Install a chart
helm upgrade myapp ./chart	Upgrade a release
helm rollback myapp 1	Rollback to revision 1

helm list	List installed releases
helm uninstall myapp	Uninstall a release
helm repo add bitnami <url>	Add a chart repository
helm show values bitnami/nginx	Display default chart values

Module 9: Amazon Web Services (AWS)

9.1 Introduction to Cloud Computing and AWS

Amazon Web Services (AWS) is the world's most comprehensive and widely adopted cloud platform, offering over 200 fully featured services from data centres globally. AWS enables businesses and developers to build scalable, reliable, and cost-effective applications without managing physical hardware.

Service Model	Description
IaaS	Infrastructure as a Service — virtual machines, storage, networking (EC2, S3)
PaaS	Platform as a Service — managed runtime environments (Elastic Beanstalk, RDS)
SaaS	Software as a Service — fully managed applications

9.2 IAM — Identity and Access Management

IAM is the security foundation of AWS. It controls who can authenticate and what actions they are authorised to perform on AWS resources.

Component	Description
Users	Individual accounts representing people or applications
Groups	Collections of users sharing the same permissions
Roles	Temporary credentials for services or cross-account access
Policies	JSON documents that define allowed/denied actions on resources
MFA	Multi-Factor Authentication adds a second layer of security

IAM Best Practices:

- Never use the root account for daily operations — create individual IAM users.
- Apply the principle of least privilege — grant only the permissions required.
- Enable MFA on all accounts, especially root and privileged users.
- Use IAM Roles for EC2 instances and Lambda functions instead of embedding access keys.
- Regularly rotate access keys and review permissions.

9.3 EC2 — Elastic Compute Cloud

EC2 provides resizable virtual servers (instances) in the cloud. It is the core compute service of AWS and is widely used for running web servers, application backends, and DevOps tools.

Instance Type	Use Case
t3.micro / t3.small	General purpose, burstable — development, low-traffic apps
m5.large / m5.xlarge	Balanced compute/memory — web servers, app servers
c5.large / c5.xlarge	Compute-optimised — CI/CD workers, batch processing
r5.large / r5.xlarge	Memory-optimised — databases, in-memory caches

Key EC2 Concepts:

- AMI (Amazon Machine Image) — a template containing the OS and pre-installed software for launching instances.
- Security Groups — virtual firewalls controlling inbound and outbound traffic to instances.
- Key Pairs — SSH key pairs for secure access to Linux instances.
- Elastic IP — a static public IPv4 address that persists even when an instance is stopped.
- Auto Scaling Groups (ASG) — automatically increase or decrease EC2 instance count based on demand.
- Elastic Load Balancer (ELB) — distributes incoming traffic across multiple EC2 instances.

9.4 S3 — Simple Storage Service

S3 is AWS's object storage service offering industry-leading scalability and durability (99.999999999%). It is used for storing files, backups, static website assets, and data lakes.

Storage Class	Use Case
S3 Standard	Frequent access — high availability and performance
S3 Intelligent-Tiering	Automatic cost optimisation by moving data between tiers
S3 Standard-IA	Infrequent access — lower cost, retrieval fee applies
S3 Glacier Instant	Archive with millisecond retrieval
S3 Glacier Deep Archive	Long-term archive — lowest cost, 12-hour retrieval

S3 Operations with AWS CLI

```
aws s3 ls                                    # List all buckets
aws s3 ls s3://my-bucket/                    # List objects in bucket
aws s3 cp backup.tar.gz s3://my-bucket/backups/  # Upload
aws s3 cp s3://my-bucket/file.txt ./         # Download
```

```
aws s3 sync ./dist/ s3://my-website-bucket/  # Sync directory
aws s3 rm s3://my-bucket/ --recursive        # Delete all objects
```

9.5 VPC — Virtual Private Cloud

VPC allows creation of a logically isolated section of the AWS cloud where resources are launched in a virtual network defined by the user.

Component	Description
VPC	Isolated virtual network; defined by a CIDR block (e.g. 10.0.0.0/16)
Subnets	Subdivisions of the VPC; can be public or private
Internet Gateway	Enables communication between VPC resources and the internet
NAT Gateway	Allows private subnet instances to access the internet without being directly reachable
Route Tables	Control the routing of network traffic within the VPC
Security Groups	Stateful instance-level firewalls
Network ACLs	Stateless subnet-level firewalls

9.6 RDS — Relational Database Service

RDS is a managed database service for setting up, operating, and scaling relational databases in the cloud. It handles routine tasks such as provisioning, patching, backup, and replication.

Supported Engines:

- MySQL, PostgreSQL, MariaDB — widely-used open-source databases.
- Oracle, Microsoft SQL Server — enterprise-grade commercial databases.
- Amazon Aurora — AWS-native with up to 5x MySQL and 3x PostgreSQL performance.

Key Features:

- Automated Backups — daily backups with point-in-time recovery for up to 35 days.
- Multi-AZ Deployments — synchronous standby replica in another AZ for high availability.
- Read Replicas — asynchronous replicas for read scaling.
- Encryption — data at rest (KMS) and in transit (SSL/TLS).

9.7 Lambda — Serverless Computing

AWS Lambda allows running code without provisioning or managing servers. It executes code in response to events such as HTTP requests (via API Gateway), S3 uploads, or scheduled triggers (via EventBridge). Lambda automatically scales from a single request to thousands per second.

```
import json

def lambda_handler(event, context):
  bucket = event['Records'][0]['s3']['bucket']['name']
  key = event['Records'][0]['s3']['object']['key']
  print(f'File uploaded: s3://{bucket}/{key}')
  return {
    'statusCode': 200,
    'body': json.dumps({'message': 'Processed successfully'})
  }
```

9.8 CloudWatch — Monitoring and Observability

Feature	Description
Metrics	Time-series data points for AWS resources (CPU, memory, network)
Logs	Centralised log management from EC2, Lambda, ECS, and other services
Alarms	Trigger notifications or actions when a metric exceeds a threshold
Dashboards	Custom visual displays of metrics and alarms
EventBridge	Rule-based event routing for automated responses to state changes

9.9 AWS CLI

```
aws configure                          # Configure credentials and region

aws ec2 describe-instances --region ap-south-1
aws ec2 start-instances --instance-ids i-1234567890abcdef0
aws ec2 stop-instances  --instance-ids i-1234567890abcdef0

aws s3 ls
aws s3 cp file.txt s3://my-bucket/

aws lambda list-functions
aws cloudwatch list-metrics --namespace AWS/EC2
```

Module 10: DevOps Culture & SRE Principles

10.1 What is DevOps?

DevOps is a cultural and organisational movement that promotes collaboration between development (Dev) and operations (Ops) teams. It aims to break down the traditional silos between these functions to enable faster, more reliable delivery of software. DevOps is not a tool or a role — it is a culture, a philosophy, and a set of practices.

10.2 The Three Ways of DevOps

First Way — Systems Thinking:

Focus on the entire system from business requirement to customer value, not just a single team's output. Optimise global performance, not local efficiencies.

Second Way — Amplify Feedback Loops:

Create fast, frequent, and high-quality feedback loops from right to left (from production back to development). This enables fast detection and correction of problems before they propagate.

Third Way — Culture of Continuous Experimentation and Learning:

Foster a culture that supports risk-taking and learning from failure. Allocate time for improvement of daily work. Practice blameless post-mortems and reward innovation.

10.3 Key DevOps Metrics — DORA

Metric	Description
Deployment Frequency	How often code is deployed to production
Lead Time for Changes	Time from code commit to running in production
Mean Time to Recovery (MTTR)	How quickly service is restored after an incident
Change Failure Rate	Percentage of deployments causing a production failure

10.4 Site Reliability Engineering (SRE)

Site Reliability Engineering (SRE) is a discipline developed at Google that applies software engineering principles to IT operations. It focuses on creating scalable and highly reliable software systems.

Concept	Description

SLI (Service Level Indicator)	A quantitative measure of service performance (e.g. availability percentage, request latency)
SLO (Service Level Objective)	A target value for an SLI (e.g. 99.9% availability over 30 days)
SLA (Service Level Agreement)	A contractual commitment to customers based on SLOs, with financial penalties for breach
Error Budget	The allowable amount of downtime or errors before the SLO is breached; governs deployment risk
Toil	Repetitive, manual, automatable operational work that scales with service growth

10.5 SRE Principles

- Embrace Risk — 100% reliability is rarely the right target. Use error budgets to balance reliability and feature velocity.
- Eliminate Toil — if an operational task is manual and repetitive, automate it.
- Monitoring and Alerting — alert on symptoms, not causes. Page on-call engineers only for actionable, urgent issues.
- Release Engineering — build reliable, repeatable deployment pipelines. Every release should be consistent and reversible.
- Simplicity — complexity is the enemy of reliability. Keep systems as simple as possible.
- Blameless Post-Mortems — when incidents occur, focus on system failures and process improvements, not individual blame.

Appendix A: Command Cheatsheet

A.1 Linux

Command	Description
ls -lah	Long listing with human-readable sizes including hidden files
chmod 755 file	rwxr-xr-x
ps aux | grep nginx	Find nginx processes
netstat -tulnp	List listening ports
df -h	Disk usage (human readable)
free -h	Memory usage (human readable)
tail -f /var/log/syslog	Follow system log
crontab -e	Edit cron jobs
systemctl status nginx	Check nginx service status

A.2 Docker

Command	Description
docker build -t app:v1 .	Build image with tag
docker run -d -p 80:80 app	Run container detached with port mapping
docker exec -it ctr bash	Shell into running container
docker compose up -d	Start all compose services
docker system prune -af	Remove all unused Docker resources
docker stats	Live container resource usage

A.3 Kubernetes

Command	Description
kubectl get all -n ns	Get all resources in a namespace
kubectl apply -f file.yaml	Apply manifest
kubectl describe pod name	Detailed pod information
kubectl logs pod -f	Follow pod logs
kubectl exec -it pod -- sh	Shell into pod
kubectl scale deploy name --replicas=3	Scale deployment

kubectl rollout undo deploy/name	Rollback deployment
kubectl top pods	Resource usage by pod

A.4 Git

Command	Description
git log --oneline --graph	Visual branch history
git cherry-pick <hash>	Apply a specific commit to current branch
git rebase -i HEAD~3	Interactive rebase last 3 commits
git reflog	History of HEAD movements (recovery tool)
git blame file.txt	Show who last modified each line of a file

A.5 AWS CLI

Command	Description
aws configure	Set up credentials and default region
aws s3 ls	List all S3 buckets
aws s3 cp file.txt s3://bucket/	Upload file to S3
aws ec2 describe-instances	List EC2 instances
aws ec2 stop-instances --instance-ids	Stop EC2 instance
aws lambda list-functions	List Lambda functions

Appendix B: Glossary

Term	Definition
AMI	Amazon Machine Image — a pre-configured virtual machine template for EC2
Artifact	A compiled or packaged output of a build process (Docker image, binary)
Blue-Green Deploy	A release strategy using two identical environments for zero-downtime deployments
Canary Deploy	Gradually routing a small percentage of traffic to a new version before full rollout
cgroups	Linux control groups — kernel feature for limiting and isolating resource usage
Container	A lightweight, portable execution environment sharing the host OS kernel
CI/CD	Continuous Integration / Continuous Delivery and Deployment
etcd	A distributed key-value store used by Kubernetes for cluster state storage
GitOps	Using Git as the single source of truth for declarative infrastructure
IaC	Infrastructure as Code — managing infrastructure through configuration files
Idempotent	An operation that produces the same result regardless of how many times applied
Kubernetes	Open-source container orchestration platform for automating deployment and scaling
Microservices	Architectural style structuring an application as independent small services
Namespace	Linux kernel isolation feature; also a K8s resource grouping mechanism
Pod	The smallest deployable unit in Kubernetes; wraps one or more containers
Rollback	Reverting to a previous version of an application or infrastructure configuration
SLA	Service Level Agreement — a contractual commitment on service availability
SLI	Service Level Indicator — a quantitative measure of service performance
SLO	Service Level Objective — a target range for an SLI
Toil	Repetitive manual operational work that scales with service growth
YAML	YAML Ain't Markup Language — human-readable format used in K8s, Docker, CI/CD

— End of Report —